

MPC 调研文档·阅读辅助手册

本文档用途：配合主调研文档使用。专为第一次接触 MPC 的读者设计，提供：

- 🎯 通俗入门（用生活化类比建立直觉）
- 📖 数学符号速查表（每个字母代表什么 + 工程取法）
- 🖼️ 文本示意图（ASCII 流程图、架构图）
- 💬 代码逐行中文注释（比主文档更详细）
- 🔗 关键概念的直觉比喻

第零章：MPC 是什么？——从"开车去机场"讲起

0.1 一个你一定经历过的场景

你开车去首都机场，手机打开导航。你会注意到：

1. 导航**永远不会**在出发前一次性告诉你"请按这个路线开 90 分钟"然后再也不说话；
2. 导航**每隔几秒**就基于你当前的 GPS 位置和未来路况预测**重新规划**一次；
3. 每次规划虽然算出了"未来 20 公里的最优路线"，但**只告诉你下个路口怎么拐**；
4. 等你真的到了下个路口（新位置）、也许前方堵车信息更新了（新预测），它**再算一次**。

这就是 MPC。你已经每天在用了。

0.2 为什么不能一次算完？

因为 **预测永远不准**。30 分钟后的路况、10 小时后的电价、明天中午的光伏出力——这些都有误差。与其相信远期的不准预测并僵化执行，不如：

- 只相信近期预测（未来 15 分钟的电价预测比未来 12 小时的电价预测准得多）；
- 基于最新测量反复重算（每 15 分钟用真实 SOC 而不是预测 SOC 作为起点）；
- 只执行最近期的决策（剩下的都可能被下一次重算覆盖）。

这三点合起来就是 **Receding Horizon Control（滚动时域控制）**，也就是 MPC 的本名。

0.3 从开车类比到 VPP 调度的一对一映射

开车导航	VPP 调度 MPC
当前位置（GPS）	当前 SOC + 机组启停状态
未来路况预测	未来 24 h 电价/PV/风电/负荷预测
路线的"最优"	成本最小 / 收益最大

开车导航	VPP 调度 MPC
红绿灯、限速、禁行	SOC 上下限、爬坡约束、功率上限
"前方 100 米右转"	"下一个 15 分钟储能放电 0.8 MW"
每几秒重算一次	每 15 min 重算一次
到达机场	一天运行结束, SOC 回到目标值

0.4 和你熟悉的"运筹优化"的关系（关键认知转换）

你熟悉这种问题：

给定 24 h 电价预测，求 BESS 的最优充放电策略，最大化套利收益，满足 SOC 约束。

这是一个标准 LP/MILP。写成伪代码：

```
result = solve_milp(price_forecast, SOC_init=0.5)
execute(result.all_24_hours)    # 一口气执行完 24 小时
```

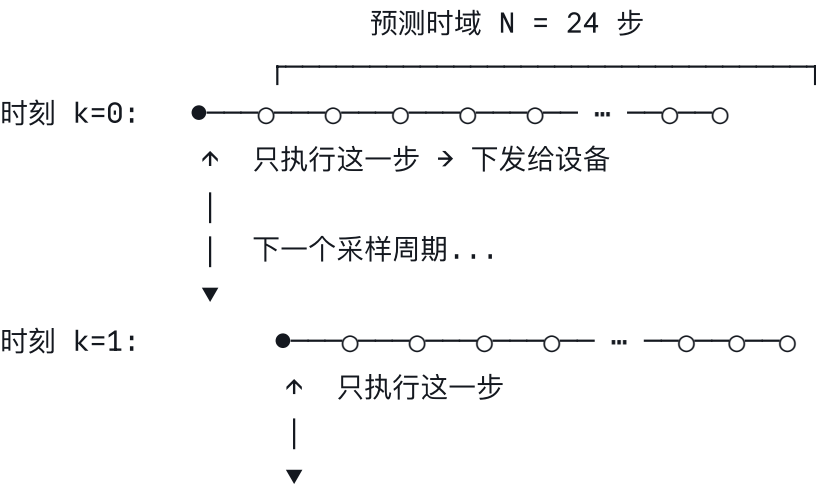
MPC 的做法是把它改成这样：

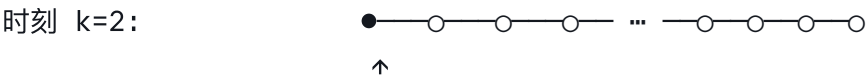
```
SOC_now = 0.5
for t in range(96):                # 每 15 分钟一次
    forecast = get_latest_forecast(t)    # ← 拿最新预测（滚动更新）
    result = solve_milp(forecast, SOC_init=SOC_now) # ← 用真实 SOC 作起点
    execute(result.first_15min)          # ← 只执行第 1 个时段
    SOC_now = measure_real_SOC()         # ← 读真实 SOC（含扰动）
```

看出来了吗？你原来就会做 MPC——你欠缺的只是那个 `for` 循环和"只执行首步"的模式。

这个认知转换大约需要 5 分钟。但真正"用好"MPC（稳定性、可行性、终端约束、经济 MPC 耗散性）需要一套理论——正是主文档第一部分展开的内容。本文档帮你把那些数学符号翻译成直觉。

0.5 滚动时域示意图（ASCII）





图例： ● = 实际执行的首步控制 ($\Delta t = 15\text{ min}$)
○ = 预测轨迹中的后续步 (被算出来但不执行)

- 核心观察：
- 1. 时域窗口 (长度 N) 随时间向右"滑动" (receding)；
 - 2. 每次重新规划用的都是"最新测量 + 最新预测"；
 - 3. 99% 的计算结果被扔掉，这看似浪费，但正是闭环鲁棒性的来源。

0.6 MPC 的"反馈"在哪里？

你可能注意到：主文档说 MPC 是"反馈控制"，但代码里看起来只有一个 `for` 循环在解优化问题，哪来的反馈？

反馈藏在每次优化的"初值"里：

`SOC_init = SOC_now` ← 这一行就是反馈！

- **开环控制**：出发时算一个 24 小时的计划，然后**按计划执行，不看反馈**（出了问题也不改）；
- **MPC**：每次用**刚测到的 SOC**（而不是上次预测的 SOC）作为初值重新规划——如果预测错了，下一次规划就会自动纠偏。

这是 MPC 对模型失配和扰动鲁棒的根本机制。没有数学，只是常识。

第一章：核心数学符号速查表（配合主文档 1.2 节）

1.1 状态空间模型

$$x_{k+1} = Ax_k + Bu_k$$

符号	含义	在 VPP 里是什么？	典型维度
k	离散时刻（第 k 步）	比如 $k = 0$ 是现在， $k = 1$ 是 15 分钟后	标量整数
x_k	状态 (state) ，系统的"内存"	SOC、机组开关、累计偏差	$n \times 1$ 列向量
u_k	控制 (control/input) ，你能决策的变量	充放电功率、机组出力、购售电量	$m \times 1$ 列向量
A	状态转移矩阵	电池自放电率（几乎为 1）	$n \times n$
B	输入矩阵	充放电效率 $\times$ 时间步长 / 容量	$n \times m$

工程记忆法： x 是"你测得到但改不了的状态"（SOC 就在那里）， u 是"你下周周期能决定的动作"（充 0.5 MW 还是放 0.5 MW）。

1.2 代价函数（二次型跟踪 MPC）

$$J_N(x_0, U) = \sum_{k=0}^{N-1} (x_k^\top Q x_k + u_k^\top R u_k) + x_N^\top P x_N$$

符号	含义	工程取法
J_N	总代价，要最小化的目标函数	——
N	预测时域（prediction horizon）	VPP 取 24h 对应步数，如 96（15min）或 24（1h）
$U = [u_0^\top, \dots, u_{N-1}^\top]^\top$	整条决策序列	优化变量
$x_k^\top Q x_k$	第 k 步的状态代价	"状态偏离理想值的惩罚"
$u_k^\top R u_k$	第 k 步的控制代价	"用力过猛的惩罚"（防止大幅抖动）
Q	状态权重矩阵（对角阵， $\succeq 0$ ）	通常 $Q = \text{diag}(q_1, q_2, \dots)$ ，对"你关心"的状态赋大权重
R	控制权重矩阵（对角阵， $\succ 0$ ）	通常 $R = \text{diag}(r_1, \dots)$ ，"防抖"权重
P	终端代价权重	取 DARE 解（离散 Riccati）以接管无穷远尾部代价
$x_N^\top P x_N$	终端代价（terminal cost）	惩罚"时域末端状态离理想太远"——防止短视

工程直觉：

- Q 大 / R 小 $\rightarrow$ 控制器激进（哪怕大幅调节也要压状态误差）；
- Q 小 / R 大 $\rightarrow$ 控制器保守（宁可状态误差大也不要频繁调节）；
- 调参本质是平衡 $\|Q\|/\|R\|$ 的比值。

1.3 约束

$$x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}, \quad x_N \in \mathcal{X}_f$$

符号	含义	VPP 中的例子
$\mathcal{X}$	状态可行域（多面体）	$\text{SOC} \in [0.1, 0.9]$
$\mathcal{U}$	输入可行域（多面体）	$ P_{\text{ch}} \leq 1 \text{ MW}$
$\mathcal{X}_f$	终端约束集（terminal set）	比如要求末端 SOC 回到 $50\% \pm 5\%$
$\in$	"属于", 代表取值必须在集合内	---

1.4 滚动优化与闭环控制

符号	含义
$U_k^* = \{u_{0 k}^*, u_{1 k}^*, \dots, u_{N-1 k}^*\}$	在时刻 k 求出的最优序列
$u_{j k}^*$	在时刻 k 规划的、用于时刻 $k + j$ 的最优控制（双下标：前一个是"预测到第几步", 后一个是"在哪个时刻规划的"）
$u_k = u_{0 k}^*$	实际下发给设备的控制（只取首步）
$\kappa_N(\cdot)$	MPC 的隐式反馈律： $u_k = \kappa_N(x_k)$
$\hat{x}_k$	状态估计（带帽子的 x , 若不能直接测量则由 Kalman/MHE 估计）

关键观察：每次求解得到的 U_k^* 里，**99% 都被扔掉了**（只用 $u_{0||k}^*$ ），剩下的 $u_{1||k}^*, \dots, u_{N-1||k}^*$ 只作为下一次求解的 warm-start。这是 MPC 的"受人诟病"但实际正确的做法。

第二章：关键概念的直觉比喻

2.1 Lyapunov 函数 = "势能"

稳定性证明里最常出现的是 **Lyapunov 函数 $V(x)$** 。它就是物理里的"势能"：

- 小球沿山坡滚下 → 势能单调下降 → 小球最终停在谷底（稳定点）；
- 闭环系统状态 x_k 沿轨迹演化 → 若 $V(x_{k+1}) < V(x_k)$ 恒成立 → 状态最终收敛到平衡点。

MPC 的稳定性证明核心套路：证明价值函数 $V_N^*(x) = \min_U J_N(x, U)$ 是 Lyapunov 函数。

2.2 控制不变集 = "安全沙盒"

控制不变集 $\mathcal{X}_f$ 的定义：

$$\forall x \in \mathcal{X}_f, \exists u \in \mathcal{U} : Ax + Bu \in \mathcal{X}_f$$

直译：只要你进了这个集合，就总有办法不出去。

类比：玩塞尔达，角色掉进河里会死（违反约束）；"控制不变集"就是地图上那些"只要你在里面就总能跳来跳去不会掉进河里"的安全平台区。

VPP 里的意义：取 $\mathcal{X}_f = \{\text{SOC} = 0.5\}$ 或 $\mathcal{X}_f = \{0.4 \leq \text{SOC} \leq 0.6\}$ 作为终端约束，含义是"时域末端必须回到一个我知道后续肯定能继续安全运行的 SOC 区间"。

2.3 递归可行性 = "骨牌推不倒"

递归可行性：若 k 时刻的优化问题有解 U_k^* ，则 $k + 1$ 时刻的优化问题也有解。

类比：多米诺骨牌。如果每张牌都能推倒下一张，就保证整个队列永远倒下去。

为什么重要？如果某一步 MPC 求解失败（不可行），就没法下发控制指令，系统会"罢工"。理论证明告诉你什么样的终端约束设计能保证永远有解。

工程弥补：实务中不是每次都严格证明理论，而是加松弛变量（软约束），保证物理上永远能返回一个"凑合用"的解。这是主文档 4.7 节"求解失败回退"的来源。

2.4 Economic MPC 的"耗散性"

标准 (Tracking) MPC 的代价是 $\|x - x_s\|^2$ （离理想点的距离平方），天然"向谷底聚拢"——这是"势能下降"直觉的来源。

Economic MPC 的代价是 $\lambda_k \cdot (P_{\text{out}} - P_{\text{in}})$ （电价 $\times$ 净售电）——这个函数没有"谷底"。电价越高，卖得越多，代价越负（收益越大）。那怎么保证稳定？

答案是 **Strict Dissipativity**（严格耗散性）：

$$\lambda(f(x, u)) - \lambda(x) \leq \ell_e(x, u) - \ell_e(x_s, u_s) - \rho(\|x - x_s\|)$$

物理直觉：存在一个"存储函数" $\lambda(x)$ （想象成电池存的"额外势能"），使得"经济代价的增长" $\geq$ "存储函数的变化量" + "偏离稳态的距离"。满足这个不等式，就能构造一个"旋转代价" $L = \ell_e + \lambda(x) - \lambda(f(x, u))$ ，它恒正——等价于一个标准 Lyapunov 函数。

VPP 工程陷阱：**SOC 没有固有耗散性**（电卖了就没了，不会自动回来）。所以 EMPC 里必须加终端 SOC 罚项 $\lambda^{\text{term}}(\text{SOC}_N - \text{SOC}^*)^2$ ，人为造一个耗散性，否则 MPC 会"把电全卖光然后无法继续运行"。

2.5 非预期约束 (Non-anticipativity) = "演员不能预知剧本"

SMPC 中的非预期约束：在不确定性揭晓之前，决策不能依赖于尚未发生的信息。

类比：假设你在拍一部实时直播的电视剧，剧情根据观众投票决定。

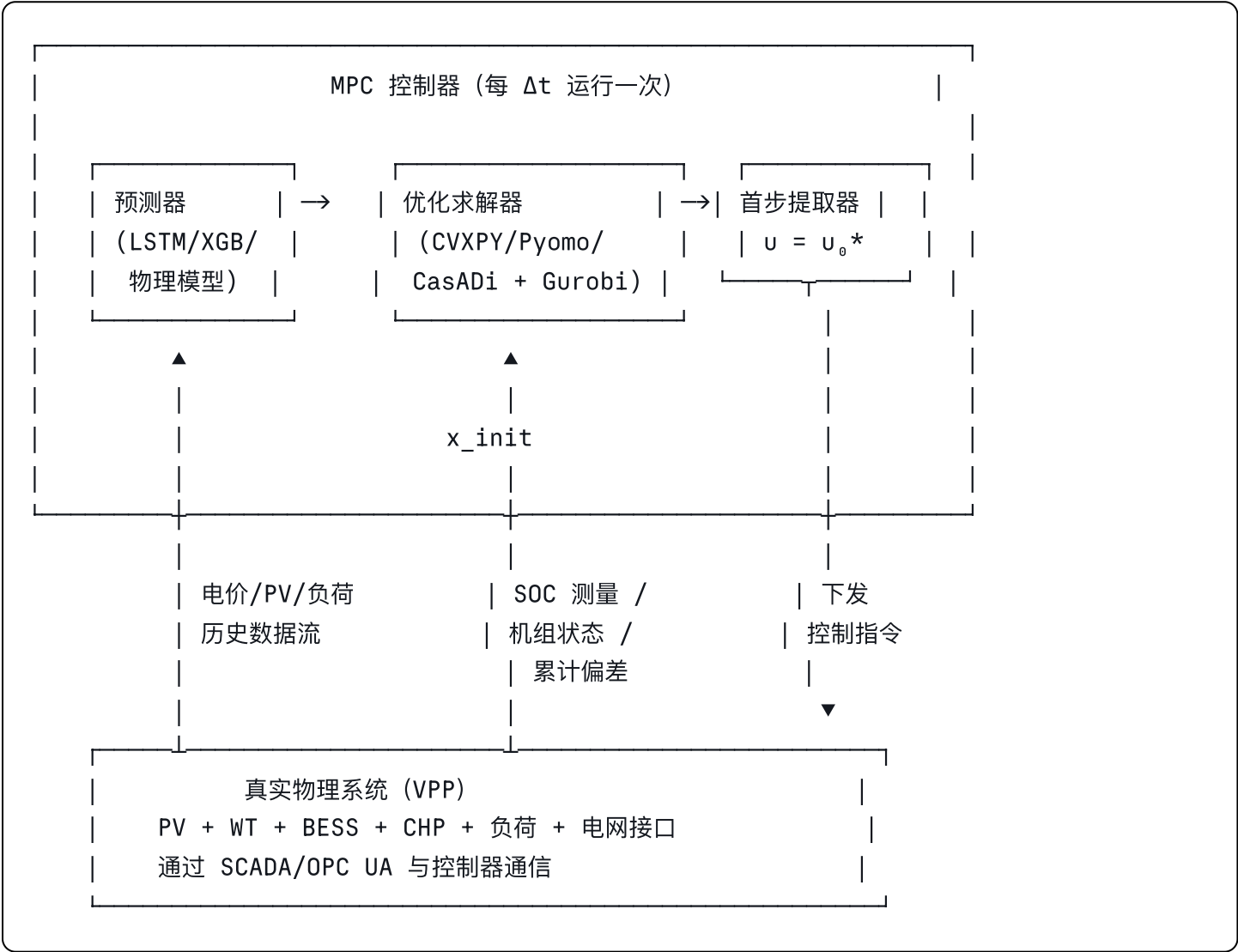
- **合法**：演员第 1 场戏的台词在所有观众投票分支中必须相同（还没投票呢）；
- **非法**：演员第 1 场戏的台词根据观众最终会怎么投而定（这是作弊）；
- **合法**：演员第 2 场戏的台词可以基于第 1 场后的投票结果而不同。

代码实现（来自主文档示例 3）：

```
python

# 非预期约束：t=0 所有场景共享同一个 u
for s in range(1, S):
    cons += [P_ch[0, s] == P_ch[0, 0]]    # 所有场景的 t=0 决策必须相同
```

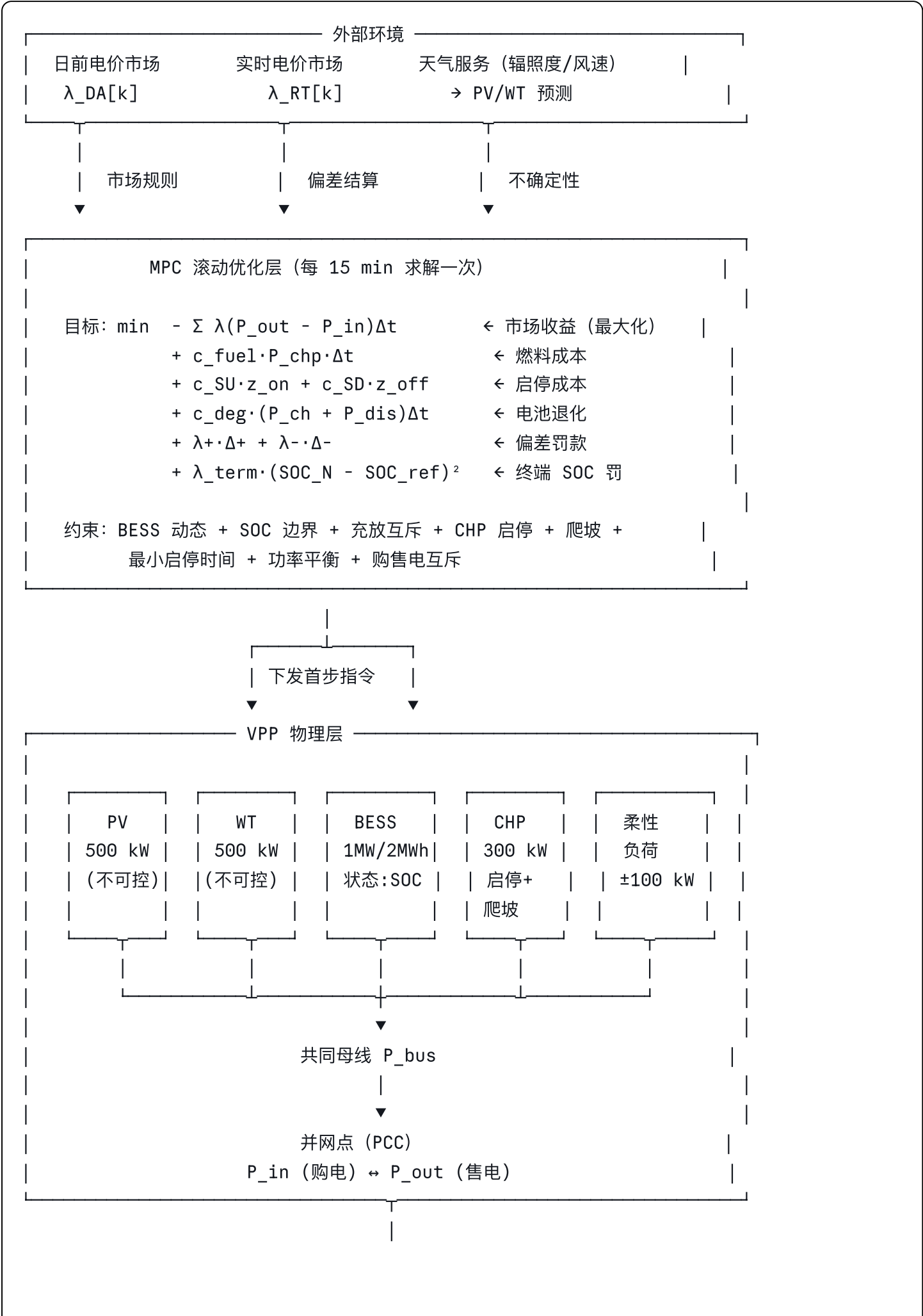
第三章：MPC 闭环架构示意图（ASCII）



三条信息流：

- **反馈流**：真实系统 → 传感器 → 状态估计 → 作为优化初值；
- **前馈流**：外部数据（电价发布、天气预报） → 预测器 → 作为优化参数；
- **控制流**：优化结果 → 首步 → 执行器 → 真实系统。

第四章：VPP 系统架构图（ASCII，配合主文档第四部分）





VPP 状态/控制变量对照表：

类别	变量名	含义	类型	典型范围
状态 x	SOC_k	电池荷电状态	连续	$[0.1, 0.9]$
	u_{k-1}^{on}	上时刻 CHP 开关状态	二元	$\{0, 1\}$
	P_{k-1}^{chp}	上时刻 CHP 出力（用于爬坡）	连续	$[0, 0.3]$ MW
控制 u	P_k^{ch}, P_k^{dis}	充/放电功率	连续	$[0, 1]$ MW
	δ_k^{ch}	充放模式选择	二元	$\{0, 1\}$
	P_k^{chp}	CHP 出力	连续	$[0, 0.3]$ MW
	u_k^{on}	CHP 本时刻开关	二元	$\{0, 1\}$
	z_k^{on}, z_k^{off}	CHP 启动/停机事件	二元	$\{0, 1\}$
	ΔP_k^{DR}	柔性负荷调节量	连续	$[-0.1, 0.1]$ MW
	P_k^{in}, P_k^{out}	购/售电量	连续	$[0, 2]$ MW
	δ_k^{buy}	买卖方向	二元	$\{0, 1\}$
	Δ_k^+, Δ_k^-	对日前承诺的正/负偏差	连续	≥ 0
	λ_k^{DA}	日前电价	连续	0.2-1.0 元/kWh
参数（外生）	$\hat{P}_k^{pv}, \hat{P}_k^{wt}$	PV/WT 预测	连续	≥ 0
	$\hat{L}_k$	负荷预测	连续	> 0
	$P_k^{bid, DA}$	日前市场承诺	连续	任意

第五章：三个代码示例的逐行详解版

5.1 示例 1：BESS 功率跟踪（CVXPY，行级注释版）

python

```

# =====
# 示例 1: 基础线性 MPC — BESS 跟踪经济调度目标功率 P_ref
# -----
# 业务场景:
#   上层系统 (EMS) 发给你一条 P_ref 曲线 (15 分钟粒度),
#   你的储能要尽可能跟踪这条曲线, 同时不能违反 SOC 和功率限。
#   典型应用: 辅助服务、AGC 跟踪、虚拟同步机模拟。
# =====

import numpy as np
import cvxpy as cp
import matplotlib.pyplot as plt

# ----- 1. BESS 物理参数 (读数据手册即可) -----
E_cap    = 4.0          # 储能总容量 4 MWh
P_max     = 1.0          # 充/放最大功率 1 MW (即 C-rate = 0.25C)
eta_c, eta_d = 0.95, 0.95 # 充/放效率 (同充同放会损失  $1 - 0.95^2 \approx 9.75\%$ )
SOC_min, SOC_max = 0.1, 0.9 # SOC 安全区间 (保护电芯寿命)
SOC_0     = 0.5          # 仿真起始 SOC (50%)
dt         = 0.25         # 控制步长 0.25 h = 15 分钟

# ----- 2. MPC 超参数 (需要调的地方) -----
N          = 24           # 预测时域 24 步 = 6 小时
                        # 取多长? 经验: 至少覆盖一个关键周期 (光伏/电价)
                        # 太短: 看不到远期, 易短视; 太长: 计算量大且远期不准
N_sim      = 96           # 闭环仿真 96 步 = 24 小时 (验证一整天)
lam_du     = 0.05         # 控制量变化率惩罚权重
                        # 大: 出力平滑 (保护电池); 小: 跟踪精度高
                        # 工程可先设 0.01 再调

# ----- 3. 构造 CVXPY 优化问题 (关键: 只构造一次!) -----
# CVXPY 的核心加速技巧: Parameter 机制
# 把"每次迭代都变"的量 (SOC 初值、P_ref 窗口) 声明为 Parameter,
# 优化问题的符号结构只编译一次, KKT 矩阵因子分解可复用。

# --- 3.1 决策变量 ---
P_ch = cp.Variable(N, nonneg=True) # 充电功率序列, 非负 (约束天然写死)
P_dis = cp.Variable(N, nonneg=True) # 放电功率序列, 非负
SOC   = cp.Variable(N + 1)          # 预测 SOC 轨迹 (N+1 个点: k=0..N)
                                        # +1 因为有 N 步后的末端状态

# --- 3.2 占位参数 (每步 MPC 更新 .value) ---
SOC_init_param = cp.Parameter() # 当前测量 SOC (反馈机制的入口!)
P_ref_param     = cp.Parameter(N) # 未来 N 步的 P_ref 预测序列
P_prev_param    = cp.Parameter() # 上一步执行的 P_net (用于 du 惩罚)

# --- 3.3 组合表达式 ---

```

```

P_net = P_dis - P_ch                                # 净出力：放电为正，充电为负

# --- 3.4 目标函数 ---
tracking_cost = cp.sum_squares(P_net - P_ref_param)
                                #  $\sum (P_{\text{net}}[k] - P_{\text{ref}}[k])^2$  跟踪误差
du_cost = cp.square(P_net[0] - P_prev_param) \
          + cp.sum_squares(P_net[1:] - P_net[:-1])
                                #  $\sum (\Delta P_{\text{net}})^2$  相邻步变化惩罚
                                # 包含从上一时刻到当前的过渡
objective = cp.Minimize(tracking_cost + lam_du * du_cost)

# --- 3.5 约束 ---
constraints = [SOC[0] == SOC_init_param] # SOC 初值固定为反馈值（核心！）

# SOC 动力学（每步递推）：
# SOC[k+1] = SOC[k] + (效率 × 充电 - 放电/效率) × dt / 容量
for k in range(N):
    constraints += [
        SOC[k+1] == SOC[k]
                        + (eta_c * dt / E_cap) * P_ch[k]      # 充电时 SOC ↑
                        - (dt / (eta_d * E_cap)) * P_dis[k]    # 放电时 SOC ↓
    ]

constraints += [
    SOC[1:] >= SOC_min,      # SOC 下限（从 k=1 开始，k=0 是测量值已定）
    SOC[1:] <= SOC_max,      # SOC 上限
    P_ch <= P_max,           # 充电功率上限
    P_dis <= P_max,          # 放电功率上限
]

# --- 3.6 把表达式、目标、约束打包成 Problem（只此一次！） ---
prob = cp.Problem(objective, constraints)
# 注意：此时还没求解，prob 只是“符号描述”

# ----- 4. 构造外部信号：24 小时的 P_ref 曲线 -----
# 实际工程中这里是上游 EMS 下发的；demo 用一个带噪正弦代替
t_axis = np.arange(N_sim + N) * dt # 多算 N 个是为了窗口能滑到末尾
P_ref_all = 0.6 * np.sin(2*np.pi*t_axis/24 - np.pi/2) \
            + 0.2 * np.sin(2*np.pi*t_axis/6)
np.random.seed(0)
P_ref_all += 0.05 * np.random.randn(len(t_axis))

# ----- 5. 闭环滚动仿真（MPC 的 "for 循环"） -----
SOC_log, P_log, Pref_log = [SOC_0], [], []
soc_now, p_prev = SOC_0, 0.0 # 初始化：真实状态 + 上一时刻控制

for t in range(N_sim):

```

```

# === 步骤 1: 更新 Parameter (每步 MPC 的反馈入口!) ===
SOC_init_param.value = soc_now          # 用最新真实 SOC
P_ref_param.value     = P_ref_all[t : t + N] # 滑动窗口取未来 N 步
P_prev_param.value    = p_prev

# === 步骤 2: 求解 (warm_start=True 复用上一次因子分解) ===
prob.solve(solver=cp.OSQP, warm_start=True)
if prob.status not in ("optimal", "optimal_inaccurate"):
    raise RuntimeError(f"MPC 求解失败: {prob.status}")
    # 工程实务: 这里要有兜底策略 (上一步解延续、降级到恒功率等)

# === 步骤 3: 只取首步! (MPC 灵魂所在) ===
p_apply = float(P_dis.value[0] - P_ch.value[0])
# 注意: 我们解出了未来 N=24 步的计划,
# 但只用第 0 步! 剩下 23 步全扔掉 (下次迭代会重新算)

# === 步骤 4: 模拟真实系统演化 ===
# demo 简化: 预测 = 真实。工程实务这里会注入测量噪声、模型失配。
if p_apply >= 0: # 放电
    soc_now = soc_now - (dt / (eta_d * E_cap)) * p_apply
else:          # 充电
    soc_now = soc_now - (eta_c * dt / E_cap) * p_apply
soc_now = float(np.clip(soc_now, SOC_min, SOC_max)) # 物理钳位
# 日志
P_log.append(p_apply); Pref_log.append(P_ref_all[t])
SOC_log.append(soc_now); p_prev = p_apply # 保存供下一步用

# ----- 6. 可视化 -----
fig, axes = plt.subplots(2, 1, figsize=(10,6), sharex=True)
axes[0].plot(np.arange(N_sim)*dt, Pref_log, 'k--', label='P_ref')
axes[0].plot(np.arange(N_sim)*dt, P_log, 'b-', label='P_BEES')
axes[0].set_ylabel('Power [MW]'); axes[0].legend(); axes[0].grid(True)
axes[1].plot(np.arange(N_sim+1)*dt, SOC_log, 'g-')
axes[1].axhline(SOC_min, ls=':', c='r'); axes[1].axhline(SOC_max, ls=':', c='r')
axes[1].set_ylabel('SOC'); axes[1].set_xlabel('Time [h]'); axes[1].grid(True)
plt.tight_layout(); plt.show()

```

示例 1 关键理解点：

- 第 3.5 节的 `SOC[0] == SOC_init_param` 是 MPC 的"反馈大门"——每次迭代都用真实测量做初值；
- 第 5 步 `P_dis.value[0] - P_ch.value[0]` 是"只执行首步"的实现；
- 效率 < 1 的性质天然阻止同充同放（能量损失使 LP 最优解不会这样）；若未来加入负电价或容量补贴，就要加整数互斥约束（下个示例）。

5.2 示例 2：电价套利 MILP（Pyomo, 行级注释版）

```
python
```

```

# =====
# 示例 2: 含整数变量的 MILP 型 MPC — BESS 电价套利
# -----
# 业务场景:
# 给你未来 24h 分时电价, BESS 低价时充电、高价时放电赚差价。
# 必须强制"充/放电不能同时发生"—引入二元变量 + Big-M。
# 这是电力市场套利最基础的模型, 也是后续 UC + ED 的起点。
# =====

import numpy as np
import pyomo.environ as pyo
import matplotlib.pyplot as plt

# ----- 参数 (与示例 1 相同, 不赘述) -----
E_cap, P_max = 4.0, 1.0
eta_c, eta_d = 0.95, 0.95
SOC_min, SOC_max, SOC_0 = 0.1, 0.9, 0.5
dt, BIG_M = 1.0, 1.0      # Big-M 取 P_max (最紧!)
                        # Big-M 是 MILP 里极重要的技巧:
                        # 约束 " $y \leq M \cdot \delta$ " 配合  $\delta \in \{0,1\}$ ,
                        # 意思是 " $\delta=0$  时  $y$  必须=0;  $\delta=1$  时  $y$  可以自由"。
                        # M 取多大? 越小越紧 (LP 松弛越紧, 分支定界越快)。
                        # 但要够大到不违反可行性—所以最紧就是  $y$  的自然上限。
N, N_sim = 24, 72        # 预测 24h; 仿真 3 天

# ----- 生成未来 4 天的电价序列 (模拟现货市场) -----
def make_price(day_idx):
    # 典型"双峰谷"电价 (早晨低、傍晚高)
    base = np.array([
        0.30, 0.28, 0.27, 0.26, 0.27, 0.30,    # 深夜-凌晨 低价 (适合充电)
        0.45, 0.70, 0.85, 0.75, 0.60, 0.55,    # 白天 中高价
        0.55, 0.60, 0.70, 0.80, 0.90, 1.00,    # 下午-晚高峰 最高价 (适合放电)
        0.95, 0.80, 0.65, 0.50, 0.40, 0.35     # 夜晚回落
    ])
    return base * (1.0 + 0.05 * np.sin(day_idx))  # 日间微扰

price_all = np.concatenate([make_price(d) for d in range(4)])

# ----- 构造 MILP 模型的函数 (每步 MPC 调用一次) -----
def build_milp(soc_init, price_window):
    """
    输入:
        soc_init      : 当前真实 SOC (反馈!)
        price_window  : 未来 N 小时的电价预测向量
    输出:
        完整的 Pyomo 模型, 可直接丢给求解器
    """

```

```

m = pyo.ConcreteModel("BESS_MILP_MPC")

# --- 时间索引集 ---
m.T = pyo.RangeSet(0, N-1) # 时间索引 0..N-1
m.Tp1 = pyo.RangeSet(0, N) # 包含终点的 0..N (用于 SOC 轨迹)

# --- 决策变量 ---
# 连续变量: 功率
m.P_ch = pyo.Var(m.T, domain=pyo.NonNegativeReals, bounds=(0, P_max))
m.P_dis = pyo.Var(m.T, domain=pyo.NonNegativeReals, bounds=(0, P_max))
# 连续变量: SOC 轨迹
m.SOC = pyo.Var(m.Tp1, domain=pyo.Reals, bounds=(SOC_min, SOC_max))
# 二元变量: 充/放电模式指示器 (核心!)
m.u_ch = pyo.Var(m.T, domain=pyo.Binary) # 1 表示该时刻允许充电
m.u_dis = pyo.Var(m.T, domain=pyo.Binary) # 1 表示该时刻允许放电

# --- 约束 ---
# (1) 初值: SOC[0] 锁定为反馈测量值
m.init = pyo.Constraint(expr=m.SOC[0] == soc_init)

# (2) SOC 动态 (与示例 1 相同公式)
m.soc_dyn = pyo.Constraint(m.T, rule=lambda m, t:
    m.SOC[t+1] == m.SOC[t]
        + (eta_c * dt / E_cap) * m.P_ch[t]
        - (dt / (eta_d * E_cap)) * m.P_dis[t])

# (3) Big-M 约束: 功率被二元变量"启用"才能非零
m.bigM_ch = pyo.Constraint(m.T, rule=lambda m, t:
    m.P_ch[t] <= BIG_M * m.u_ch[t])
# 当 u_ch=0 时: P_ch ≤ 0 → P_ch = 0 (因为下界是 0)
# 当 u_ch=1 时: P_ch ≤ P_max (自然约束)
m.bigM_dis = pyo.Constraint(m.T, rule=lambda m, t:
    m.P_dis[t] <= BIG_M * m.u_dis[t])

# (4) 充放互斥 (SOS1 约束的线性写法)
m.excl = pyo.Constraint(m.T, rule=lambda m, t:
    m.u_ch[t] + m.u_dis[t] <= 1)
# 含义: 同一时刻 u_ch 和 u_dis 最多一个为 1 (另一个必为 0)
# → P_ch 和 P_dis 不能同时非零 → 物理上不同充同放

# (5) 目标函数: 最大化套利收益
# 收益 = Σ (电价 × 净售电量) = Σ price × (P_dis - P_ch) × dt
m.obj = pyo.Objective(
    expr=sum(price_window[t] * (m.P_dis[t] - m.P_ch[t]) * dt for t in m.T),
    sense=pyo.maximize
)

return m

```

```

# ----- 求解器自动发现 -----
def get_solver():
    """按优先级尝试 MILP 求解器"""
    for name in ("appsi_highs", "highs", "cbc", "glpk"):
        try:
            s = pyo.SolverFactory(name)
            if s.available(exception_flag=False):
                print(f"[solver] 使用 {name}")
                return s
        except:
            continue
    raise RuntimeError("未找到 MILP 求解器, 请 pip install highs 或 coinor-cbc")

solver = get_solver()

# ----- 闭环 MPC 滚动 -----
SOC_log, Pch_log, Pdis_log, price_log = [SOC_0], [], [], []
soc_now = SOC_0

for t in range(N_sim):
    # 1. 取未来 N 步电价 (滚动窗口)
    window = price_all[t : t + N]

    # 2. 构建并求解 MILP
    model = build_milp(soc_now, window)
    res = solver.solve(model, tee=False)
    if res.solver.termination_condition != pyo.TerminationCondition.optimal:
        raise RuntimeError(f"MILP 在时刻 {t} 求解失败")

    # 3. 只执行首步
    pch_0 = pyo.value(model.P_ch[0])
    pdis_0 = pyo.value(model.P_dis[0])

    # 4. 状态更新 (demo: 预测 = 真实)
    soc_now = float(np.clip(
        soc_now
        + (eta_c * dt / E_cap) * pch_0
        - (dt / (eta_d * E_cap)) * pdis_0,
        SOC_min, SOC_max
    ))

    # 5. 日志
    SOC_log.append(soc_now)
    Pch_log.append(pch_0); Pdis_log.append(pdis_0)
    price_log.append(window[0])

```

```
# ----- 收益统计 -----  
revenue = sum(price_log[t] * (Pdis_log[t] - Pch_log[t]) * dt  
              for t in range(N_sim))  
print(f"[result] {N_sim}h 累计套利收益 = {revenue:.2f} 元")
```

示例 2 关键理解点：

- **Big-M 法**把"逻辑 if-then"转成线性约束，是 MILP 工程化的万能工具；
- `build_milp` 每步重新构建稍微慢，更好的工程做法是一次性构建模型，每步只更新 `m.SOC[0].fix(new_value)` 和价格参数——Pyomo 的 `mutable` 参数机制支持；
- **求解器兼容**：HiGHS（开源最强）→ CBC → GLPK，任何一个可用即可。

5.3 示例 3：场景 Stochastic MPC（CVXPY，行级注释版）

```
python
```

```

# =====
# 示例 3: 场景 Stochastic MPC (SMPC)
# -----
# 业务场景:
#   负荷和电价"较准" (短期预测误差小),
#   但光伏 PV 有预测误差—特别是云层遮挡造成的突变。
#   我们不用单点预测, 而是生成 S=20 个等概率场景,
#   对"期望购电成本"最小化。
# -----
# 核心技术点:
#   (a) 多场景决策变量: 形状 (N, S) 的张量
#   (b) 非预期约束: t=0 时所有场景决策必须相同
#   (c) 期望代价 =  $\sum \pi_s \cdot \text{Cost}_s$ 
# =====

import numpy as np
import cvxpy as cp
import matplotlib.pyplot as plt

# ----- 基本参数 -----
E_cap, P_max = 4.0, 1.0
eta_c, eta_d = 0.95, 0.95
SOC_min, SOC_max, SOC_0 = 0.1, 0.9, 0.5
dt, N, N_sim, S = 0.25, 16, 80, 20
# N=16 步 = 4h 预测; S=20 个场景; 仿真 N_sim=80 步 = 20h
np.random.seed(42)

# ----- 生成"真实"时间序列 (用于评估) -----
total_steps = N_sim + N
t_axis = np.arange(total_steps) * dt
# 真实光伏 (正弦 + 日出日落截断)
PV_true = np.maximum(0, 1.2 * np.sin(np.pi * (t_axis - 6) / 12))
PV_true[(t_axis < 6) | (t_axis > 18)] = 0
# 真实负荷
Load_true = 0.6 + 0.3 * np.sin(2*np.pi*t_axis/24 - np.pi/2)
# 真实电价 (假设可完美预测)
price = 0.4 + 0.3 * np.sin(2*np.pi*t_axis/24 - 1.0)

# ----- 场景生成函数 -----
def sample_pv_scenarios(t_start):
    """
    以真实 PV 为均值, 加入相对误差, 生成 S 个 PV 预测场景。
    核心思想: 预测越远的时刻, 误差越大 ( $\sigma$  随步数线性增长)。
    实际工程会用:
    - 分位数回归 (QR) 的离散化
    - Copula / GAN 生成模型
    - 历史类比日残差抽样
    """

```

```

"""
mean = PV_true[t_start : t_start + N]
sigma = 0.10 + 0.02 * np.arange(N) # 时变标准差：越远越不准
noise = np.random.randn(N, S) * sigma[:, None] # (N, S)
scenarios = np.maximum(0, mean[:, None] * (1.0 + noise)) # 非负截断
probs = np.ones(S) / S # 等概率假设
return scenarios, probs

# ----- 多场景 CVXPY 建模 -----
# 关键：决策变量是 (N, S) 的 2D 张量
# 每列代表某个场景下的决策序列
P_ch = cp.Variable((N, S), nonneg=True)
P_dis = cp.Variable((N, S), nonneg=True)
SOC = cp.Variable((N + 1, S))
P_grid = cp.Variable((N, S)) # 购电为正，售电为负

# Parameters
SOC_init_p = cp.Parameter()
PV_p = cp.Parameter((N, S)) # 每列是一个 PV 场景
Load_p = cp.Parameter(N) # 负荷在所有场景相同（简化）
price_p = cp.Parameter(N)
pi_p = cp.Parameter(S, nonneg=True) # 场景概率

# --- 约束 ---
cons = []

# (a) 每个场景内的 SOC 动态
for s in range(S):
    cons += [SOC[0, s] == SOC_init_p] # 初值（所有场景相同）
    for k in range(N):
        cons += [SOC[k+1, s] == SOC[k, s]
                  + (eta_c * dt / E_cap) * P_ch[k, s]
                  - (dt / (eta_d * E_cap)) * P_dis[k, s]]

# (b) SOC 和功率的边界约束
cons += [
    SOC[1:, :] >= SOC_min,
    SOC[1:, :] <= SOC_max,
    P_ch <= P_max,
    P_dis <= P_max,
]

# (c) 功率平衡（每个场景独立）
# 购电 + PV + 放电 - 充电 = 负荷
for s in range(S):
    cons += [
        P_grid[:, s] + PV_p[:, s] + P_dis[:, s] - P_ch[:, s] == Load_p
    ]

```

```

    ]

# (d) 非预期约束 (SMPC 灵魂!)
# t=0 的决策在揭晓不确定性之前, 必须在所有场景下相同
for s in range(1, S):
    cons += [
        P_ch[0, s] == P_ch[0, 0],
        P_dis[0, s] == P_dis[0, 0],
    ]
# 注意 t=1, 2, ... 不施加此约束—那时 PV 实现已揭晓, 可以依场景不同

# --- 目标: 期望代价 ---
# 每个场景的购电成本
cost_per_scn = cp.sum(
    cp.multiply(price_p[:, None] * dt, P_grid),    # element-wise
    axis=0
) # shape (S,)

# 期望成本 =  $\sum \pi_s \cdot \text{cost}_s$ 
expected_cost = cost_per_scn @ pi_p

# 正则项 (防止数值病态, 很小的权重)
reg = 1e-3 * (cp.sum_squares(P_ch) + cp.sum_squares(P_dis)) / S

prob = cp.Problem(cp.Minimize(expected_cost + reg), cons)

# ----- 闭环仿真 -----
SOC_log, Papp_log, Pgrid_log = [SOC_0], [], []
soc_now = SOC_0

for t in range(N_sim):
    # 1. 生成场景 (工程中由概率预测模型给出)
    pv_scn, pi = sample_pv_scenarios(t)
    PV_p.value, pi_p.value = pv_scn, pi
    Load_p.value = Load_true[t : t + N]
    price_p.value = price[t : t + N]
    SOC_init_p.value = soc_now

    # 2. 求解
    prob.solve(solver=cp.ECOS)

    # 3. 只取首步 (所有场景首步都相同, 所以取 s=0 即可)
    p_ch_0 = float(P_ch.value[0, 0])
    p_dis_0 = float(P_dis.value[0, 0])
    p_bess = p_dis_0 - p_ch_0

    # 4. 用真实 PV 演化状态 (关键差异!)

```

```

soc_now = float(np.clip(
    soc_now + (eta_c * dt / E_cap) * p_ch_0
    - (dt / (eta_d * E_cap)) * p_dis_0,
    SOC_min, SOC_max
))
p_grid_real = Load_true[t] - PV_true[t] - p_bess # 用真实 PV!
# 注意：这里体现了 SMPC 的价值 — 我们的决策考虑了 PV 不确定性,
# 但实际执行时用的是真实 PV 值。若预测过于乐观（场景集偏差大），
# 实际购电量会与期望不同 — SMPC 只能保证统计意义上最优。

SOC_log.append(soc_now)
Papp_log.append(p_bess)
Pgrid_log.append(p_grid_real)

total_cost = sum(price[t] * Pgrid_log[t] * dt for t in range(N_sim))
print(f"[result] SMPC 真实购电成本 = {total_cost:.2f} 元")

```

示例 3 关键理解点：

- 非预期约束是 SMPC 区别于朴素多场景求解的核心，没有它就是"每个场景单独最优"而不是"面对不确定性的最优鲁棒决策"；
- 场景数 S 与计算量线性相关，工程中 $S=10-30$ 常见，超过 50 后建议做场景缩减（Dupačová 2003 方法）；
- 扩展 CVaR 风险约束：引入 ζ 与 $\eta_s \geq 0$ 辅助变量即可把机会约束写成线性。

第六章：VPP 综合案例代码的分块解读（配合主文档 4.6 节）

VPP 综合案例的 Pyomo 代码较长，下面把它拆成 6 个"概念块"，每块用一句话概括功能：

块 1：参数与外部数据	→ "所有数字都从这里读，便于你改来做 A/B 实验"
块 2：时间索引与决策变量	→ "描述'你在哪些时刻能做什么决定'"
块 3：BESS 约束集	→ "电池这一类设备的所有物理规律"
块 4：CHP 约束集（含启停时间）	→ "燃气机组的逻辑约束：最麻烦的部分"
块 5：功率平衡 + 市场互斥	→ "VPP 内外的能量守恒"
块 6：Economic MPC 目标函数	→ "把'赚钱'翻译成数学"

6.1 块 4 详解：Rajan-Takriti 最小启停时间

为什么这个约束值得单独讲？因为 UC（机组组合）是电力系统最核心的 MILP 建模，而这个紧式表达又是 UC 建模的教科书标准。

业务背景：燃气轮机一旦启动，因为热应力等原因必须运行至少 T^{up} 个时段才能再停机；停机后也需要冷却至少 T^{dn} 才能再启动。

数学表达 (Rajan-Takriti 2005) :

$$\sum_{\tau=k-T^{\text{up}}+1}^k z_{\tau}^{\text{on}} \leq u_k^{\text{on}}$$

直觉解读: "过去 T^{up} 个时段内每一次启动事件, 都要求当前时刻机组处于开状态"。

- 若 $k = 10, T^{\text{up}} = 4$: 左边求和是 $z_7^{\text{on}} + z_8^{\text{on}} + z_9^{\text{on}} + z_{10}^{\text{on}}$;
- 若过去四步任何一步启动过 (至少一个 $z^{\text{on}} = 1$), 则 $u_{10}^{\text{on}} \geq 1$, 即现在必须是开着的 ;
- 反过来, 如果 $u_{10}^{\text{on}} = 0$ (现在关着), 则过去四步都不能有启动事件。

****为什么叫"紧式"**: 这个写法比朴素的 $u_{k+1}^{\text{on}} + u_{k+2}^{\text{on}} + \dots \geq T^{\text{up}} \cdot z_k^{\text{on}}$ 的 LP 松弛更紧, 实际求解快 2-5 倍。做 UC 的工程师应该刻进骨子里。**

6.2 终端 SOC 罚项为何不可省？

回忆第 2.4 节的讨论：**EMPC + 积分器状态（SOC）必须显式加终端罚。**

直觉：如果你不管终端 SOC，MPC 会算出"当前这 24h 把电全部卖了最赚钱"——因为没有人替它考虑"明天早上怎么办"。加一个 $\lambda^{\text{term}}(\text{SOC}_N - \text{SOC}^{\text{ref}})^2$ 相当于给它植入一条"末端必须回到 50%，否则罚钱"的规则——强迫它为明天保留电量。

工程建议： λ^{term} 取 100-1000 元/kWh² 级别，使终端罚的量级与 24h 总收益相当即可（一天几千元级别）。取值过小约束无效，过大会挤占套利收益。可做灵敏度扫描。

第七章：学习路线与练手建议

7.1 对"已经会运筹优化、想掌握 MPC"的工程师的路线

阶段	任务	预计时间
理解	读本文档 + 主文档第一部分	2-4 h
上手	跑通三个代码示例，改改参数观察行为	4-8 h
消化	理解终端约束、递归可行性理论	3-5 h
应用	仿照 VPP 综合案例改造自己的业务代码	2-5 天
进阶	加入 MHE 状态估计、场景缩减、软约束	1-2 周
生产	OPC UA 接入、回退策略、容器化部署	2-4 周

7.2 容易踩的坑清单

1. 忘加终端约束/罚项 → EMPC 把 SOC 全卖光；
2. Big-M 取得太大 → LP 松弛太松, MILP 求解慢 10 倍；
3. 预测窗口 N 取太短 → "短视"决策, 错失远期机会；
4. Parameter 机制没用好 → 每步重新编译 CVXPY, 速度慢 5-10 倍；
5. 求解失败无兜底 → 生产系统随机罢工；
6. 用预测值演化状态 → 仿真结果过于乐观, 上线惊吓；
7. 场景数 S 太多 → SMPC 求解时间爆炸；
8. 爬坡约束忘了处理 $k=0$ 的上一时刻 → 首次启动报错。

7.3 推荐练手题

- 入门：示例 1 基础上, 把"跟踪 P_{ref} "改成"最小化用电费用 (给定负荷和分时电价)"——把 VPP 最基础的电费优化建出来；
- 中阶：示例 2 基础上, 加入电池退化成本 $c^{deg} \cdot (P^{ch} + P^{dis})$, 观察套利量的变化；
- 进阶：示例 3 基础上, 把 PV 的独立场景改为 PV + 负荷相关场景 (用 copula 或多维 AR)；加入 CVaR 风险约束对比期望代价与 CVaR-优化的决策差异；
- 实战：用你熟悉的电力市场数据 (如广东现货、山东现货) 跑完整 VPP 案例, 对比"离线 LP"和"滚动 MPC"的盈利差距——这通常是 15-30% 的差异, 直接是项目落地的 ROI 依据。

本阅读辅助文档为主调研文档的配套, 建议两份搭配阅读：本文档用于建立直觉、速查符号、逐行理解代码；主文档用于深入理论、参考文献、工程落地细节。